MANUAL DIDÁCTICO

EVENTOS EN jQuery 3.7.1

Explicaciones · Ejemplos HTML completos · Prácticas · Ejercicios resueltos

Objetivo del manual: aprender a detectar las acciones del usuario y responder a ellas mediante eventos de ratón, teclado, formularios y navegador; registrar, eliminar y disparar manejadores; y controlar el objeto event y la propagación.

Material preparado para trabajar directamente en clase

Índice y organización

1. Qué es un evento y cómo funciona jQuery
2. Preparación del entorno y $(document).ready()
3. Eventos de ratón
4. Eventos de teclado
5. Eventos de formularios
6. Eventos del navegador y carga de recursos
7. Registrar, ejecutar y eliminar manejadores: on(), one(), off(), trigger() y triggerHandler()
8. El objeto event
9. Propagación, delegación y acciones predeterminadas
10. Métodos obsoletos o eliminados: qué evitar
11. Proyecto integrador: panel de interacción

Versión de referencia: jQuery 3.7.1. Los ejemplos utilizan HTML, CSS y JavaScript estándar junto con jQuery.

Cómo utilizarlo: cada ejemplo completo puede copiarse en un archivo .html y abrirse en un navegador. Se necesita conexión a Internet para cargar jQuery desde el CDN indicado; también se puede descargar la biblioteca y cambiar la ruta del script.

1. Qué es un evento

2. Preparación del entorno y ready()

jQuery debe cargarse antes de ejecutar código que utilice $. Además, los elementos HTML que seleccionamos deben existir cuando se ejecuta el código.

2.1. ¿Qué hace $(document).ready()?

```
$(document).ready(function() {
  // El árbol DOM ya está construido.
});
```

La función se ejecuta cuando el navegador ha terminado de construir el DOM. No espera necesariamente a que se descarguen todas las imágenes, hojas de estilo u otros recursos.

2.2. Forma abreviada

```
$(function() {
  console.log("El DOM está preparado");
});
```

2.3. ¿Hace falta ready() al final de body?

Si el script está situado al final de body, después de los elementos HTML que utiliza, normalmente no es necesario ready(): el navegador ya ha procesado esos elementos antes de ejecutar el script.

```
<body>
  <button id="accion">Pulsa</button>

  <script>
    // El botón ya existe en el DOM.
    $("#accion").on("click", function() {
      alert("Acción ejecutada");
    });
  </script>
</body>
```

Regla práctica: script en head: utiliza ready() o defer. Script al final de body: normalmente puedes registrar los manejadores directamente.

3. Eventos de ratón

Los eventos de ratón permiten responder a clics, movimientos y entradas o salidas del puntero.

Evento	Cuándo ocurre	Uso habitual
click	Al hacer clic (pulsar y soltar) sobre un elemento.	Botones, enlaces y controles.
dblclick	Al hacer doble clic.	Acciones especiales de edición o selección.
mouseenter	Cuando el puntero entra en el elemento.	Resaltar una tarjeta al entrar.
mouseleave	Cuando el puntero sale del elemento.	Restaurar el aspecto al salir.
mouseover	Cuando el puntero entra en el elemento o en uno de sus descendientes.	Detectar entrada con propagación.
mouseout	Cuando el puntero sale del elemento hacia otro elemento o descendiente.	Detectar salida con propagación.
mousedown	Al presionar un botón del ratón.	Detectar el inicio de una pulsación.
mouseup	Al soltar un botón del ratón.	Detectar el final de una pulsación.
mousemove	Mientras se mueve el puntero.	Coordenadas, dibujo o seguimiento.
contextmenu	Al solicitar el menú contextual, normalmente con botón derecho.	Personalizar o impedir el menú contextual.

3.1. Diferencia entre mouseenter/mouseleave y mouseover/mouseout

mouseenter y mouseleave no se disparan por entrar o salir de los elementos hijos. mouseover y mouseout sí pueden dispararse al pasar entre el elemento y sus descendientes. Esta diferencia se aprecia especialmente cuando una caja contiene otra.

3.2. Ejemplo completo: laboratorio de ratón

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <title>Eventos de ratón</title>
 <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
 <style>
  body { font-family: Arial; padding: 20px; }
  #caja { padding: 25px; background: #dceefa; border: 2px solid #246; }
  #interior { margin-top: 15px; padding: 20px; background: white; }
  .activo { background: #ffe69a !important; }
  #registro { min-height: 100px; border: 1px solid #aaa; padding: 10px; }
 </style>
</head>
<body>
 <h1>Prueba los eventos de ratón</h1>
 <div id="caja">
  Caja exterior
  <div id="interior">Caja interior</div>
 </div>
 <p id="texto">Haz clic, doble clic o usa el botón derecho sobre este texto.</p>
 <div id="registro">Los eventos aparecerán aquí.</div>

 <script>
  function registrar(mensaje) {
   $("#registro").prepend("<p>" + mensaje + "</p>");
  }

  $("#caja").on("mouseenter", function() {
   $(this).addClass("activo");
```

```
    registrar("mouseenter: el puntero ha entrado en la caja exterior");
  });

  $("#caja").on("mouseleave", function() {
    $(this).removeClass("activo");
    registrar("mouseleave: el puntero ha salido de la caja exterior");
  });

  $("#caja").on("mouseover", function() {
    registrar("mouseover: entrada en la caja o en un descendiente");
  });

  $("#caja").on("mouseout", function() {
    registrar("mouseout: salida de la caja o de un descendiente");
  });

  $("#texto").on("click", function() {
    registrar("click: clic sencillo");
  });
  $("#texto").on("dblclick", function() {
    registrar("dblclick: doble clic");
  });
  $("#texto").on("mousedown", function() {
    registrar("mousedown: botón del ratón presionado");
  });
  $("#texto").on("mouseup", function() {
    registrar("mouseup: botón del ratón soltado");
  });
  $("#texto").on("mousemove", function(event) {
    registrar("mousemove: X=" + event.pageX + " Y=" + event.pageY);
  });
  $("#texto").on("contextmenu", function(event) {
    event.preventDefault();
    registrar("contextmenu: menú contextual bloqueado en este texto");
  });
 </script>
</body>
</html>
```

Práctica guiada: mueve el ratón entre la caja exterior y la interior. Compara cuántos mensajes producen mouseenter/mouseleave y mouseover/mouseout. Después comenta temporalmente los manejadores para comprobar qué cambia.

4. Eventos de teclado

Los eventos de teclado permiten detectar la pulsación y liberación de teclas. Se utilizan, por ejemplo, para validar entradas, crear atajos o actualizar una vista mientras el usuario escribe.

keydown: Se produce cuando se presiona una tecla.

keyup: Se produce cuando se libera una tecla.

Sobre keypress: no se incluye en los ejemplos de código nuevo. Es un evento antiguo y su comportamiento no es adecuado para detectar todas las teclas. Para aplicaciones actuales utiliza keydown y keyup, y consulta event.key.

4.1. Ejemplo completo

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <title>Eventos de teclado</title>
 <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>
 <h1>Detector de teclado</h1>
 <label for="tecla">Escribe en este campo:</label>
 <input id="tecla" type="text">
 <p id="salida">Esperando una tecla...</p>

 <script>
  $("#tecla").on("keydown", function(event) {
   $("#salida").text("keydown: has pulsado " + event.key);
  });

  $("#tecla").on("keyup", function(event) {
   $("#salida").text("keyup: has soltado " + event.key);
  });
 </script>
</body>
</html>
```

event.key devuelve la tecla interpretada por el navegador, por ejemplo a, Enter, Escape o ArrowUp. Es preferible a códigos numéricos antiguos.

4.2. Ejemplo: atajo de teclado

```
$(document).on("keydown", function(event) {
 if (event.ctrlKey && event.key.toLowerCase() === "s") {
  event.preventDefault();
  alert("Atajo Ctrl + S detectado");
 }
});
```

El ejemplo detecta Ctrl+S y evita que el navegador abra el cuadro de guardado. En una aplicación real, el manejador debería guardar los datos de la aplicación.

5. Eventos de formularios

Los formularios contienen campos que reciben el foco, pierden el foco, cambian de valor o se envían. jQuery permite reaccionar a cada una de estas acciones.

Evento	Cuándo ocurre	Observación
focus	Un control recibe el foco.	Se aplica al elemento enfocado.
blur	Un control pierde el foco.	Se aplica al elemento que pierde el foco.
focusin	Un elemento o descendiente recibe el foco.	Se propaga; útil en contenedores.
focusout	Un elemento o descendiente pierde el foco.	Se propaga; útil en contenedores.
change	El valor cambia y el cambio se confirma.	En texto suele confirmarse al perder el foco; en select, al cambiar opción.
select	El usuario selecciona texto dentro de un campo compatible.	No es lo mismo que seleccionar una opción de un select.
submit	Se intenta enviar un formulario.	Puede cancelarse con preventDefault().

5.1. Ejemplo completo: validación básica

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <title>Eventos de formularios</title>
 <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
 <style>
  input { display: block; margin: 8px 0 14px; padding: 7px; }
  .aviso { color: #b00020; }
  .correcto { color: #16733a; }
 </style>
</head>
<body>
 <h1>Registro de alumno</h1>
 <form id="registro">
  <label for="nombre">Nombre:</label>
  <input id="nombre" type="text" required>

  <label for="correo">Correo:</label>
  <input id="correo" type="email" required>

  <label for="curso">Curso:</label>
  <select id="curso" required>
   <option value="">Selecciona...</option>
   <option>DAW</option>
   <option>DAM</option>
   <option>ASIR</option>
  </select>

  <button type="submit">Enviar</button>
 </form>
 <p id="mensaje"></p>

 <script>
  $("#nombre").on("focus", function() {
   $("#mensaje").text("Escribe tu nombre");
  });
  $("#nombre").on("blur", function() {
   if ($(this).val().trim() === "") {
    $("#mensaje").text("El nombre no puede estar vacío");
   }
```

```
  });

  $("#registro").on("focusin", "input, select", function() {
   $(this).css("background-color", "#fff8d6");
  });
  $("#registro").on("focusout", "input, select", function() {
   $(this).css("background-color", "");
  });

  $("#curso").on("change", function() {
   $("#mensaje").text("Curso seleccionado: " + $(this).val());
  });

  $("#nombre").on("select", function() {
   $("#mensaje").text("Has seleccionado parte del texto del nombre");
  });

  $("#registro").on("submit", function(event) {
   event.preventDefault();
   const nombre = $("#nombre").val().trim();
   const correo = $("#correo").val().trim();

   if (nombre === "" || correo === "") {
    $("#mensaje").attr("class", "aviso").text("Completa los campos obligatorios.");
    return;
   }
   $("#mensaje").attr("class", "correcto")
    .text("Formulario validado para " + nombre + ". Envío simulado.");
  });
 </script>
</body>
</html>
```

Diferencia esencial: submit se registra en el elemento form, no en el botón. event.preventDefault() cancela el envío real para que podamos validar o procesar los datos mediante JavaScript.

6. Eventos del navegador y carga de recursos

Evento	Cuándo ocurre	Elemento habitual
resize	Cambia el tamaño de la ventana.	window
scroll	Se desplaza la página o un contenedor.	window o contenedor desplazable
load	Se ha cargado un recurso o la página y sus recursos dependientes.	window, img y otros recursos
error	Se produce un error al cargar un recurso.	img, script y otros recursos

El evento load no es equivalente a ready(). ready() indica que el DOM está construido; load indica que también han terminado de cargarse los recursos asociados al objeto correspondiente.

6.1. Ejemplo completo

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <title>Eventos del navegador</title>
 <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
 <style>
  body { min-height: 1400px; font-family: Arial; }
  #zona { height: 100px; overflow: auto; border: 1px solid; }
 </style>
</head>
<body>
 <h1>Eventos de ventana y recursos</h1>
 <p id="estado">Estado inicial</p>
 <div id="zona">
  <p>Contenido 1</p><p>Contenido 2</p><p>Contenido 3</p>
  <p>Contenido 4</p><p>Contenido 5</p><p>Contenido 6</p>
 </div>
 <img id="foto" src="https://www.w3.org/Icons/w3c_home" alt="Recurso de ejemplo">

 <script>
  $(window).on("resize", function() {
   $("#estado").text("Ventana: " + window.innerWidth + " px de ancho");
  });

  $(window).on("scroll", function() {
   console.log("Desplazamiento de página: " + window.scrollY);
  });

  $("#zona").on("scroll", function() {
   console.log("Desplazamiento dentro de la caja: " + this.scrollTop);
  });

  $("#foto").on("load", function() {
   $("#estado").text("La imagen se ha cargado correctamente");
  });

  $("#foto").on("error", function() {
   $("#estado").text("No se ha podido cargar la imagen");
  });
 </script>
</body>
</html>
```

Nota de prueba: para comprobar error, cambia temporalmente la ruta de la imagen por una dirección inexistente. El evento load/error debe asociarse antes de que el recurso termine de cargarse; por eso se registra en el script posterior al elemento y se recomienda controlar el orden de carga.

7. Gestión de eventos

7.1. on(): registrar un manejador

```
$("#boton").on("click", function() {
  console.log("Manejador registrado");
});
```

on() es el método general recomendado. Permite registrar uno o varios eventos, y también delegarlos desde un contenedor.

7.2. one(): ejecutar una sola vez

```
$("#aviso").one("click", function() {
  alert("Este mensaje solo aparece una vez");
});
```

7.3. off(): eliminar manejadores

```
function mostrarMensaje() {
  console.log("Se ha pulsado el botón");
}

$("#boton").on("click", mostrarMensaje);
$("#quitar").on("click", function() {
  $("#boton").off("click", mostrarMensaje);
});
```

Para poder eliminar un manejador concreto, conviene guardar la función en una variable con nombre. Si se registra una función anónima y no se conserva una referencia, resulta más difícil retirarla de forma selectiva.

7.4. trigger(): disparar un evento

```
$("#boton").trigger("click");
```

trigger() provoca el evento en jQuery y ejecuta sus manejadores. Puede participar en la propagación y en la secuencia de acciones predeterminadas asociadas.

7.5. triggerHandler(): ejecutar manejadores sin propagación

```
$("#boton").triggerHandler("click");
```

triggerHandler() ejecuta los manejadores del primer elemento seleccionado, pero no propaga el evento y no ejecuta la acción predeterminada del navegador. No equivale a simular una interacción física real.

7.6. Ejemplo completo de gestión

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <title>Gestión de eventos</title>
 <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>
 <h1>Gestión de manejadores</h1>
 <button id="principal">Botón principal</button>
 <button id="unaVez">Solo una vez</button>
 <button id="quitar">Quitar evento principal</button>
 <button id="trigger">Ejecutar trigger</button>
 <button id="triggerHandler">Ejecutar triggerHandler</button>
 <p id="resultado">Esperando...</p>

 <script>
  function accionPrincipal() {
   $("#resultado").text("Se ejecutó el manejador principal");
  }

  $("#principal").on("click", accionPrincipal);

  $("#unaVez").one("click", function() {
   alert("Este manejador se ejecuta una sola vez");
  });

  $("#quitar").on("click", function() {
   $("#principal").off("click", accionPrincipal);
   $("#resultado").text("Manejador principal eliminado");
  });

  $("#trigger").on("click", function() {
   $("#principal").trigger("click");
  });

  $("#triggerHandler").on("click", function() {
   $("#principal").triggerHandler("click");
  });
 </script>
</body>
</html>
```

8. El objeto event

Cuando se produce un evento, el manejador puede recibir un objeto event. Este objeto ofrece información sobre el origen, el tipo de evento, la posición del puntero y el comportamiento que podemos controlar.

Propiedad o método	Significado	Uso
event.type	Nombre del evento.	Saber si fue click, keydown, etc.
event.target	Elemento donde se originó el evento.	Identificar el elemento pulsado.
event.currentTarget	Elemento cuyo manejador se está ejecutando.	Consultar el elemento manejado.
event.delegateTarget	Elemento donde se registró el manejador delegado.	Identificar el contenedor delegado.
event.pageX / event.pageY	Coordenadas del puntero respecto al documento.	Posición del ratón.
event.key	Tecla asociada a eventos de teclado.	Detectar Enter, Escape, etc.
event.metaKey	Indica si se mantiene pulsada la tecla Meta.	Atajos dependientes del sistema.
event.relatedTarget	Elemento relacionado con la entrada o salida del puntero.	Comparar origen/destino.
event.timeStamp	Marca temporal del evento.	Mediciones aproximadas.
event.data	Datos adicionales pasados al registrar el manejador.	Enviar contexto al listener.
event.namespace	Espacio de nombres del evento, si se utiliza.	Organizar y retirar manejadores.
event.result	Último valor devuelto por un manejador.	Comunicación limitada entre manejadores.
event.preventDefault()	Cancela la acción predeterminada.	Evitar navegación o envío.
event.stopPropagation()	Detiene la propagación hacia ancestros.	Evitar que el padre reciba el evento.
event.stopImmediatePropagation()	Detiene la propagación y los manejadores restantes del mismo elemento.	Cortar ejecución posterior.
event.isDefaultPrevented()	Indica si se canceló la acción predeterminada.	Comprobar preventDefault().
event.isPropagationStopped()	Indica si se detuvo la propagación.	Comprobar stopPropagation().
event.isImmediatePropagationStopped()	Indica si se detuvo la propagación inmediata.	Comprobar stopImmediatePropagation().

La propiedad which se considera obsoleta en favor de propiedades más específicas, como event.key para teclas y event.button/event.buttons para botones del ratón.

8.1. Ejemplo completo: inspeccionar event

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <title>Objeto event</title>
 <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>
 <h1>Información del evento</h1>
 <button id="analizar">Pulsa este botón</button>
 <input id="teclado" placeholder="Pulsa una tecla">
 <pre id="datos"></pre>

 <script>
  $("#analizar").on("click", function(event) {
   $("#datos").text(
    "type: " + event.type + "\n" +
    "target: " + event.target.id + "\n" +
    "currentTarget: " + event.currentTarget.id + "\n" +
    "pageX: " + event.pageX + "\n" +
    "pageY: " + event.pageY + "\n" +
    "timeStamp: " + event.timeStamp
   );
  });

  $("#teclado").on("keydown", function(event) {
   $("#datos").text("Tecla: " + event.key);
  });
 </script>
</body>
</html>
```

9. Propagación, delegación y acciones predeterminadas

9.1. Propagación

Por defecto, muchos eventos se propagan desde el elemento donde se originan hacia sus elementos antecesores. Si un botón está dentro de un div, al pulsar el botón pueden ejecutarse tanto el manejador del botón como el del div.

9.2. stopPropagation()

```
$("#hijo").on("click", function(event) {
 event.stopPropagation();
 console.log("Se ejecuta el hijo, pero no el manejador del padre");
});
```

El método stopPropagation() de JavaScript sirve para detener la propagación de un evento por el árbol de elementos HTML (DOM). Es decir, evita que un evento que se produce en un elemento hijo continúe propagándose hacia sus elementos padres.

9.3. stopImmediatePropagation()

Además de impedir que el evento siga subiendo por el árbol DOM, evita que se ejecuten otros manejadores posteriores registrados para el mismo elemento.

9.4. preventDefault()

Cancela la acción predeterminada del navegador, pero no detiene por sí mismo la propagación. Por ejemplo, puede impedir que un enlace navegue a otra página o que un formulario se envíe.

9.5. Delegación de eventos

La delegación consiste en registrar el manejador en un elemento contenedor y especificar un selector para los elementos descendientes. Es especialmente útil cuando los elementos interiores se crean dinámicamente después de cargar la página.

```
$("#lista").on("click", "li", function(event) {
 $(this).toggleClass("seleccionado");
});
```

9.6. Ejemplo completo: delegación y propagación

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <title>Delegación y propagación</title>
 <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
 <style>
  .caja { padding: 20px; margin: 10px; border: 2px solid #246; }
  .seleccionado { background: #ffe69a; }
 </style>
</head>
<body>
 <h1>Delegación y propagación</h1>
 <div id="lista" class="caja">
  <button class="elemento">Elemento A</button>
  <button class="elemento">Elemento B</button>
 </div>
 <button id="crear">Crear otro botón</button>
 <div id="padre" class="caja">
  Padre
  <button id="hijo">Pulsa el hijo</button>
 </div>
 <p id="salida"></p>

 <script>
  // Delegación: también funcionará con botones añadidos más tarde.
  $("#lista").on("click", ".elemento", function() {
   $(this).toggleClass("seleccionado");
   $("#salida").text("Pulsado: " + $(this).text());
  });

  $("#crear").on("click", function() {
   $("#lista").append('<button class="elemento">Nuevo elemento</button>');
  });

  $("#padre").on("click", function() {
   $("#salida").text("Se ha ejecutado el manejador del padre");
  });

  $("#hijo").on("click", function(event) {
   event.stopPropagation();
   $("#salida").text("Se ha pulsado el hijo; la propagación se ha detenido");
  });
 </script>
</body>
</html>
```

10. Métodos obsoletos o eliminados

En jQuery moderno se recomienda utilizar on() y off(). Algunos métodos antiguos fueron eliminados en versiones anteriores a 3.7.1 y otros quedaron obsoletos antes de su eliminación.

Método antiguo	Situación	Alternativa actual
.live() / .die()	Eliminados.	Delegación mediante .on() / .off().
.error() como atajo de evento	Eliminado como método abreviado.	.on('error', manejador).
.load() como atajo de evento	Eliminado como método abreviado; no confundir con AJAX .load().	.on('load', manejador) cuando proceda.
.unload() como atajo de evento	Eliminado/no recomendado.	Evitar depender de unload; para necesidades de ciclo de vida estudiar pagehide/visibilitychange según el caso.
.bind() / .unbind()	Obsoletos.	.on() / .off().
.delegate() / .undelegate()	Obsoletos.	Delegación con .on() / .off().
.click(), .focus(), .submit(), etc. como registro/disparo	Atajos obsoletos en jQuery 3.7.1.	.on('click', ...) y .trigger('click') según la intención.
.toggle() como alternancia de manejadores	Antigua forma eliminada.	Gestionar estado con clases/estado y .on().
keypress	Evento antiguo y no recomendado para código nuevo.	keydown / keyup y event.key.
event.which	Propiedad obsoleta en favor de propiedades específicas.	event.key, event.button o event.buttons.

Precisión importante: no todos los nombres de evento han desaparecido. Por ejemplo, click sigue siendo un evento válido; lo obsoleto es utilizar el atajo .click() para registrar o disparar manejadores. En código nuevo utiliza .on('click', ...) y .trigger('click') cuando quieras dispararlo programáticamente.

El método .load() merece especial atención: jQuery conserva .load(url, ...) como método AJAX, pero eliminó el antiguo atajo de evento .load(handler). Para el evento load utiliza .on('load', handler) sobre el recurso correspondiente.

11. Proyecto integrador: panel de interacción

En esta práctica se reúnen los contenidos principales. El alumnado construirá una página con una zona de ratón, un campo de teclado, un formulario, botones para gestionar eventos y un registro de actividad.

11.1. Código HTML completo

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Panel de eventos jQuery</title>
 <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
 <style>
  body { font-family: Arial, sans-serif; max-width: 900px; margin: 25px auto; padding: 0 15px; }
  section { border: 1px solid #bbb; border-radius: 8px; padding: 15px; margin: 15px 0; }
  button, input, select { padding: 8px; margin: 5px; }
  #zonaRaton { background: #eaf4fb; padding: 25px; border: 2px solid #246; }
  #registro { min-height: 150px; max-height: 260px; overflow: auto; background: #f5f5f5; padding: 10px; }
  .activo { background: #ffe69a !important; }
 </style>
</head>
```

```
<body>
 <h1>Panel de eventos jQuery 3.7.1</h1>

 <section>
  <h2>Ratón</h2>
  <div id="zonaRaton">Haz clic, doble clic y mueve el ratón por esta zona.</div>
 </section>

 <section>
  <h2>Teclado</h2>
  <input id="campoTeclado" placeholder="Escribe aquí">
 </section>

 <section>
  <h2>Formulario</h2>
  <form id="formulario">
   <input id="nombre" placeholder="Nombre" required>
   <select id="curso" required>
    <option value="">Selecciona curso</option>
    <option>DAW</option><option>DAM</option><option>ASIR</option>
   </select>
   <button type="submit">Enviar</button>
  </form>
 </section>

 <section>
  <h2>Gestión de eventos</h2>
  <button id="botonGestion">Botón gestionado</button>
  <button id="quitar">Quitar manejador</button>
  <button id="disparar">Disparar con trigger()</button>
  <button id="soloUnaVez">Solo una vez</button>
 </section>

 <section>
  <h2>Registro</h2>
  <button id="limpiar">Limpiar</button>
  <div id="registro">Los eventos aparecerán aquí.</div>
 </section>

 <script>
  function registrar(texto) {
   if ($("#registro").text() === "Los eventos aparecerán aquí.") {
    $("#registro").empty();
   }
   $("<p>").text(texto).prependTo("#registro");
  }

  $("#zonaRaton")
   .on("mouseenter", function() {
    $(this).addClass("activo");
    registrar("mouseenter: entrada en la zona");
   })
   .on("mouseleave", function() {
    $(this).removeClass("activo");
    registrar("mouseleave: salida de la zona");
   })
   .on("click", function() {
    registrar("click: clic en la zona");
   })
   .on("dblclick", function() {
    registrar("dblclick: doble clic en la zona");
   })
   .on("mousemove", function(event) {
    registrar("mousemove: " + event.pageX + ", " + event.pageY);
   });

  $("#campoTeclado").on("keydown", function(event) {
```

```
    registrar("keydown: " + event.key);
  }).on("keyup", function(event) {
    registrar("keyup: " + event.key);
  });

  $("#formulario").on("submit", function(event) {
    event.preventDefault();
    registrar("submit: formulario de " + $("#nombre").val() +
          " para " + $("#curso").val());
  });

  function accionGestionada() {
    registrar("Se ejecutó el manejador del botón gestionado");
  }

  $("#botonGestion").on("click", accionGestionada);
  $("#quitar").on("click", function() {
    $("#botonGestion").off("click", accionGestionada);
    registrar("Manejador eliminado con off()");
  });
  $("#disparar").on("click", function() {
    $("#botonGestion").trigger("click");
    registrar("Se solicitó trigger('click')");
  });
  $("#soloUnaVez").one("click", function() {
    registrar("Manejador one(): ejecutado una vez");
  });
  $("#limpiar").on("click", function() {
    $("#registro").empty();
  });
 </script>
</body>
</html>
```